

**KSCSC**

Cyber Security Consulting · Offensive Security & Cloud

Security Assessment — eCommerce Platform

Anonymised work sample · web, admin portal & cloud API review

Assessment	eCommerce platform security assessment
Prepared by	KSCSC — Krzysztof Świdrak
Client	Redacted (direct-sales retailer)
Approach	Black/grey-box · web, admin portal & cloud API
Classification	Confidential — anonymised work sample

This is a redacted work sample. Client identity, brand, hostnames, cloud account and internal host identifiers, third-party vendor names and personal data have been replaced with neutral placeholders. Proof-of-concept requests are retained to show technique. Nothing in this document identifies the original engagement or its subjects.

Contents

Summary	4
Part 1 — eCommerce Shop (`shop.example`)	5
Non-functional WAF protection	5
SQL injection (API)	5
Brute force on representative password reset	5
Password spraying	6
UILogger captures plaintext passwords	6
Weak password policy	6
Missing backend validation of payment method	6
Cookies without proper security flags	6
Exposing backend services	6
Overly permissive `Access-Control-Allow-Origin`	7
Vulnerable JavaScript components	7
Missing security headers	7
Email and personal data embedded in a JS script	7
Missing validation of the account ID on URL password change	8
Improper input validation	8
Weak cryptography	8
Missing DNSSEC	8
Part 2 — Privileged Admin Portal (`admin.example`)	9
Privileged portal publicly exposed to the internet	9
Non-functional WAF protection	9
Brute force on login	9
Login user enumeration	9
Vulnerable JavaScript components	9
Weak password policy	9
Missing Content-Security-Policy	10
Internal server exposure after unexpected inputs	10
Missing DNSSEC	10
Part 3 — Cloud API & Infrastructure	11
Unresolved cloud security recommendations	11
WAF not blocking requests	11

Missing origin access control between CDN and API gateway	11
Internal load-balancer DNS resolvable publicly	11
Third-party components note	12

Security Assessment — eCommerce Platform

Summary

This assessment covered a consumer eCommerce shop operated by a direct-sales retailer ("the Client") and its associated privileged administration portal, together with the supporting cloud API layer. Testing was performed over a two-week window under a black/grey-box approach.

The review identified vulnerabilities across several classes:

- **Authentication weaknesses** — brute force on password resets, password spraying, weak password policies, and login user enumeration.
- **Data-protection issues** — plaintext passwords captured in UI logs, personal data embedded in JavaScript, and improper cookie security flags.
- **Input validation** — insufficient validation of user inputs and URL parameters, and a confirmed SQL injection on the API layer.
- **Missing security headers** — no CSP, Permissions-Policy, or Strict-Transport-Security, and overly permissive CORS.
- **Backend exposure** — publicly exposed internal services and identifiers, non-functional WAF protection, and a privileged portal exposed to the internet.
- **Vulnerable components** — outdated JavaScript libraries loaded into users' browsers.
- **Infrastructure** — unresolved cloud security recommendations, inadequate API-gateway origin control, and internal DNS resolvable publicly.

Overall the weaknesses range from authentication flaws and weak data protection to exposure of backend services and misconfigured infrastructure, several of which could contribute to data breaches or account compromise.

Anonymisation notice. This is a redacted work sample. The client identity, brand, hostnames, cloud account identifiers, internal host names, third-party vendor names, personal data, and representative IDs have been replaced with neutral placeholders (shop.example, api.example, admin.example, <repId>, etc.). Proof-of-concept requests are retained to show technique. Nothing here identifies the original engagement or its subjects.

Part 1 — eCommerce Shop (shop.example)

Scope: the shop front-end shop.example and its API api.example.

Non-functional WAF protection

Both services appear to be protected by a WAF, yet none of the malicious requests were blocked. A basic, well-known XSS payload in the URL passed straight through:

```
GET https://shop.example/test?q=<script>alert()</script>
GET https://api.example/v1/PL/PL/rep/<script>alert()</script>
GET https://api.example/v1/PL/PL/rep/<repId>/password?q=%3Cscript%3Ealert()%3C/script%3E
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

XSS payload passing the WAF; backend headers (server/nginx) returned directly

None of the malicious requests were blocked — responses (404/400/405) hit backend services directly, as the leaked backend headers show. Over two weeks of testing the WAF neither blocked nor alerted.

Recommendation. Consolidate CDN and WAF onto a single, properly tuned platform managed by a dedicated team, rather than running two loosely-configured WAF layers.

SQL injection (API)

```
GET /v1/PL/PL/rep/<repId>/orderTracking?q=%7B%22fltrs%22%3A%5B%7B%22fld%22%3A%22...%22%2C%22co
ndtn%22%3A%22eq%22%2C%22valTxt%22%3A%22NEW%22%7D%5D%7D HTTP/2
Host: api.example
```

The valTxt field inside the q JSON parameter is vulnerable to SQL injection; an automated tool confirmed boolean-based blind SQLi. This could allow database exfiltration and potentially access to internal, on-premise services.

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Automated tool confirming boolean-based blind SQLi on the valTxt parameter

Recommendation. Enable the WAF to block malicious SQL, and ensure all inputs are parameterised and sanitised before reaching the database.

Brute force on representative password reset

```
POST /v1/PL/PL/rep/password/reset HTTP/2
Host: api.example
{ "sndgChnl": "EMAIL", "userId": "<EMAIL | 8-digit account id>" }
```

The endpoint requires no authentication by design and enforces no anti-brute-force controls. Response codes differ by account state (204 valid-with-email / 400 valid-without-email / 404 no-account), enabling enumeration of all valid representative account IDs. In one 15-second run, 2000+ requests enumerated valid IDs. An attacker could then deliberately lock every representative's account by submitting three wrong passwords per ID.

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Intruder run distinguishing valid (204) / valid-no-email (400) / non-existent (404) account IDs

Recommendation. Configure quotas and throttling for critical endpoints, deploy a functional WAF, add a CAPTCHA, and require confirmation that the email belongs to the representative before generating a password.

Password spraying

```
POST /pl-home/api/mab/secure/access HTTP/2
Host: shop.example
{ "userId": "<ID>", "password": "<PASSWD>" }
```

No CAPTCHA or WAF blocked automated spraying; 12,000+ requests were sent without any blockade.

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

12,000+ spraying requests sent with no throttling or blocking

Recommendation. Quotas/throttling on critical endpoints, functional WAF, CAPTCHA.

UI logger captures plaintext passwords

On a website error, the UI logger transmits the plaintext password, risking credential leakage if a user mistypes their password.

Recommendation. Exclude credential fields from logging, or anonymise them.

Weak password policy

Representatives can set trivial passwords (e.g. 12345678 is accepted). This enables the password-spraying class above.

Recommendation. Enforce a modern password policy in line with current guidance.

Missing backend validation of payment method

```
GET /v1/PL/PL/rep/<repId>/order/<orderId>/pymt?dlvryTyp=<type>&dlvryId=<id> HTTP/2
Host: api.example
```

The response describing available payment methods can be modified client-side, letting a user set a disabled method (e.g. cash-on-delivery) as active for delivery types where it should be blocked:

```
{"cash":{"stus":"DISABLED"},...}  ->  {"cash":{"stus":"ACTIVE"},...}
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Modified response enabling a payment method that should be disabled for the delivery type

Recommendation. Validate all parameters server-side, not only in the frontend.

Cookies without proper security flags

The session cookies (avn_tkn — a JWT used to authorise API calls — and avn_uid) lack HttpOnly and SameSite, making them readable by JavaScript. Combined with the missing CSP, an XSS or supply-chain issue could let an attacker steal these cookies and impersonate users.

Recommendation. Apply proper cookie flags and design the session model securely.

Exposing backend services

All API responses disclose backend infrastructure detail — server software, an APM injection header, internal node hostnames, environment name, API name and version, CDN cache identifiers and edge-location codes:

```
Server: nginx
X-<apm>-Js-Injection: true
X-Node-Id: <internal-host>.<internal-domain>
X-Instance-Name: prod
X-API-Name: <internal API name>
X-API-Version: <ver>
Via: 1.1 <hash>.<cdn> (CDN)
X-<cdn>-Cf-Pop: <edge-code>
X-<cdn>-Cf-Id: <request-id>
```

Four internal application hosts were disclosed (also externally scanned by a vulnerability scanner, which found numerous issues), and these hosts process requests from the public internet.

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

API response headers disclosing internal node hostnames, environment, API name/version and CDN identifiers

Recommendation. Do not expose backend services. Strip all internal information at the public perimeter (WAF/CDN edge).

Overly permissive Access-Control-Allow-Origin

Both `api.example` and `shop.example` return `Access-Control-Allow-Origin: *`. This can facilitate credential/token theft via malicious sites, or generate unwanted cross-origin traffic (cost or DoS).

Recommendation. Restrict to trusted origins by rewriting the `Origin` against an allow-list.

Vulnerable JavaScript components

The shop loads components with confirmed vulnerabilities (multiple end-of-life jQuery versions, an EoL Bootstrap, and an outdated date library), all delivered to users' browsers.

Recommendation. Update to the latest supported versions.

Missing security headers

The services do not set **Content-Security-Policy**, **Permissions-Policy**, or **Strict-Transport-Security**. Each absence removes a browser-side protection (against XSS/injection, feature abuse, and TLS-stripping MITM respectively).

Recommendation. Define a restrictive CSP loading content only from trusted sources; set a least-privilege Permissions-Policy; enforce HSTS with a defined max-age.

Email and personal data embedded in a JS script

A production JavaScript bundle contained a hard-coded sample customer object with a real name, email address (a third-party contractor's), postal address and phone number embedded in the source.

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Personal data (name, email, address, phone) embedded in a production JS bundle — values redacted

Recommendation. Run SAST scanners before publishing content to production to eliminate data leakage. (*Specific values redacted in this sample.*)

Missing validation of the account ID on URL password change

```
PUT /v1/PL/PL/rep/<Id>/password HTTP/2
Host: api.example
```

Requests are authorised by a JWT, but the endpoint accepts any ID in the URL path, which can cause confusion or misinterpretation of logs.

Recommendation. Validate the URL ID against the JWT claim, or remove the ID from the URL.

Improper input validation

The API does not consistently validate or sanitise inputs, leaving it exposed to injection classes (SQLi, XSS, command injection).

Recommendation. Validate every input against an allow-list with default-deny; use SAST/DAST before release.

Weak cryptography

The platform stores MD5 password hashes, and some TLS 1.2 cipher suites are considered weak.

Recommendation. Improve the HTTPS/TLS configuration to strong ciphers only, and migrate password storage to a modern KDF.

Missing DNSSEC

shop.example and api.example do not have DNSSEC configured, leaving the domains exposed to spoofing/tampering.

Recommendation. Configure DNSSEC.

Part 2 — Privileged Admin Portal (`admin.example`)

Scope: the admin portal `admin.example`.

The portal grants privileged access to the shop and to customer and representative profiles: it can impersonate a representative, change their password, create orders as them, and change their email. Impersonation operations carry a dedicated token identifying the acting admin.

Recommendations (portal-level). Monitor all activity (only errors are logged today); deny creating orders and changing email while impersonating; deny password changes without an enforced policy.

Privileged portal publicly exposed to the internet

The admin portal is publicly reachable at `https://admin.example/...` Exposing privileged, administrative interfaces to the public internet significantly increases the risk of brute force, privilege escalation, and data breach.

Recommendation. Isolate privileged portals to an intranet / trusted segments, or restrict to trusted public sources only.

Non-functional WAF protection

```
GET https://admin.example/login.page?q=%3Cscript%3Ealert()%3C/script%3E
```

The XSS payload was not blocked; over two weeks the WAF neither blocked nor alerted.

Brute force on login

```
POST /login.page HTTP/1.1
Host: admin.example
_csrf=<token>&userIdDisplay=admin&password=<PWD>&x=0&y=0
```

No CAPTCHA or lockout policy; a wordlist attack on the admin username ran 7000 requests without disruption.

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

7000-request wordlist attack on the admin login with no lockout

Recommendation. Quotas/throttling, functional WAF, CAPTCHA, and a proper lockout policy.

Login user enumeration

The same login endpoint returns distinct messages for a non-existent account ("account number is not valid") versus a wrong password ("password is not valid"), enabling user enumeration.

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Different responses for non-existent account vs. wrong password

Recommendation. Unify the message to "the password or username is incorrect".

Vulnerable JavaScript components

The portal loads an end-of-life jQuery version into users' browsers.

Recommendation. Update to the latest supported version.

Weak password policy

The portal allows short/simple passwords for both admin (e.g. a 6-char value) and, via the portal, for representative accounts (as short as two characters). This enables spraying.

Recommendation. Enforce a strong password policy on both admin and representative accounts.

Missing Content-Security-Policy

The portal does not set a CSP, increasing XSS and data-injection risk.

Recommendation. Define a policy loading content only from trusted sources.

Internal server exposure after unexpected inputs

Unhandled endpoints and special-character inputs (e.g. %@!) return raw backend (Java) error messages to the frontend, disclosing sensitive internal detail.

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Raw backend Java error surfaced to the frontend after unexpected input

Recommendation. Sanitise error messages; log detail server-side only; strip internal information at the perimeter.

Missing DNSSEC

admin.example does not have DNSSEC configured.

Recommendation. Configure DNSSEC.

Part 3 — Cloud API & Infrastructure

Scope: the API management layer `api.example` (cloud account redacted).

The public-facing platform is hosted on a cloud account with a multi-layered schema. The WAF attached to the CDN was not even monitored — its logs were enabled only at the start of the testing window. Reverse-proxy logs were limited: only URL and method were useful, and request origin could not be traced because every field was overwritten by the CDN or API gateway.

Unresolved cloud security recommendations

The cloud provider's own advisory and posture-management services listed multiple **unresolved** security recommendations and findings against the account.

Recommendation. Triage and remediate the provider's security-advisory and posture findings on a defined schedule.

WAF not blocking requests

A non-functional WAF exposes the API to increased attacks (no protection from SQLi/XSS), unfiltered traffic (downtime / breach risk), and compliance gaps. WAF statistics from the testing week confirmed no effective blocking.

Recommendation. Consolidate onto a single, properly-configured CDN+WAF managed by a dedicated team.

Missing origin access control between CDN and API gateway

Without an authoriser or origin-access binding between the CDN and the API gateway, the API can be reached directly, bypassing the WAF entirely — enabling unauthorised access, public exposure, and increased attack risk. Direct access bypassing the CDN was confirmed.

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

API gateway reached directly, bypassing the CDN/WAF

Recommendation. Enforce a Zero-Trust, identity-first model: allow only CDN-originated traffic to the gateway (IAM policy or a custom header), ideally via private origins and a private gateway endpoint. Authorise gateway-to-backend connections (e.g. mTLS).

Internal load-balancer DNS resolvable publicly

An internal IP is resolvable by public DNS resolvers.

Recommendation. Implement split-horizon DNS so internal names are kept private, reducing the external attack surface and segregating internal from public queries.

Third-party components note

During testing, numerous third-party sites and components interacted with the browser (via redirects, dynamically-loaded content, and analytics/marketing scripts). *(The full inventory is redacted from this sample as it identifies the environment.)*

Recommendation. The service provider should monitor and control the third-party sites and components involved in delivering its services.